LLM Tokenizer 的算法有哪些？

Tokenizer（分词器）是大语言模型把文本转换成 Token ID 的组件。

例如：

```
输入文本：
Hello, world! 你好，世界！

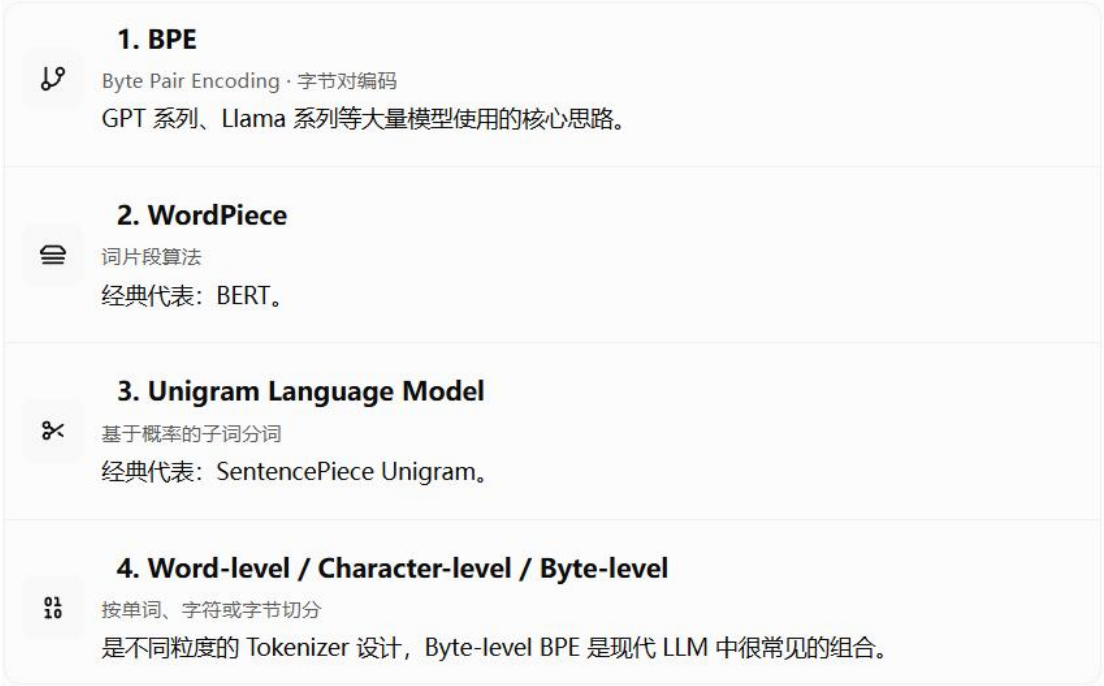
Tokenizer：
["Hello", ",", " world", "!", " 你", "好", "，", "世界", "!"]

Token IDs：
[15496, 11, 995, ...]
```

实际切分结果取决于使用的 Tokenizer，不同模型可能完全不同。

一、主流 Tokenizer 算法

目前 LLM 中最重要的有以下几类：



二、BPE：最值得重点掌握的算法

BPE 是你做 LLM、RAG、Embedding、Token 计费时最应该理解的算法之一。

1. 基本思想

从最小单位开始，不断合并出现频率高的相邻 Token。

例如，训练语料中经常出现：

```
low
lower
lowest
```

初始字符：

```
l o w e r
```

统计相邻字符频率，发现 `l + o` 经常出现：

```
lo w e r
```

继续合并：

```
low e r
```

最终可能得到：

```
["low", "er"]
```

这只是示意，真实 BPE 的合并规则由训练语料决定。

2. BPE 的训练流程

```
大量训练文本
      ↓
初始化词表（字符或字节）
      ↓
统计相邻 Token 对的频率
      ↓
合并最高频 Token 对
      ↓
更新词表和语料
      ↓
重复合并
      ↓
得到最终 Vocabulary + Merge Rules
```

3. BPE 的优点

词表大小可控。

常见词和词片段可以被一个 Token 表示。

未登录词可以拆成更小的 Token。

适合英语及多语言文本。

训练和推理效率较好。

三、Byte-level BPE：GPT 类模型常见方案

Byte-level BPE 在 BPE 的基础上使用字节作为初始单位。

例如 UTF-8:

```
A        → 1 个字节
你       → 3 个字节
😀       → 4 个字节
```

Byte-level BPE 可以处理任意 Unicode 文本，不需要为每一个字符预先建立完整词表。

典型代表包括:

GPT-2：Byte-level BPE。

GPT-3：使用 GPT-2 风格 Tokenizer。

GPT-4：使用不同的现代 Tokenizer 设计，不能简单认为与 GPT-2 完全相同。

注意：Byte-level BPE 不等于所有 GPT 模型都使用同一种 Tokenizer。

四、WordPiece：BERT 使用的算法

WordPiece 和 BPE 很相似，也是把词拆成子词。

例如:

```
playing
```

可能切分为:

```
play ##ing
```

其中 `##ing` 表示它是接在前一个子词后面的片段。

WordPiece 与 BPE 的主要区别:

特征	BPE	WordPiece

核心思想	频率驱动合并	基于词表概率/评分选择子词
常见代表	GPT-2、Llama 系列中的 BPE 变体	BERT
训练目标	合并规则	选择高价值子词
未登录词处理	继续拆分	继续拆分或使用 `[UNK]`，取决于实现

五、Unigram：SentencePiece 的另一条路线

Unigram 与 BPE 的区别很重要。

BPE

从小到大合并：

```
a b c d
  ↓
ab c d
  ↓
ab cd
  ↓
abcd
```

Unigram

从一个较大的候选词表开始，通过概率模型选择最合适的切分。

例如：

```
unbelievable
```

候选切分可能包括：

```
["un", "believable"]
["un", "believe", "able"]
["u", "n", "believable"]
```

Unigram 根据模型学习到的概率，选择整体得分更好的切分。

常见代表：

SentencePiece Unigram。

T5 的 SentencePiece Tokenizer。

六、SentencePiece 是算法吗？

严格来说，SentencePiece 是一个 Tokenizer 库 / 框架，不是单独的一种分词算法。

它支持多种模型，包括：

- BPE
- Unigram

例如：

```
import sentencepiece as spm
sp = spm.SentencePieceProcessor(
    model_file="tokenizer.model"
)
tokens = sp.encode(
    "Hello 你好世界",
    out_type=str
)
print(tokens)
```

可能输出：

```
['▁Hello', '▁你', '好', '世界']
```

其中 ▁ 是 SentencePiece 常见的空格标记。

七、其他 Tokenizer 类型

1. Word-level

按单词切分：

```
I love machine learning
["I", "love", "machine", "learning"]
```

问题：词表非常大，遇到新词会产生 OOV（Out-of-Vocabulary）。

2. Character-level

按字符切分：

```
你好
["你", "好"]
```

优点是可以处理新词，缺点是 Token 数量通常更多。

3. Byte-level

按字节切分：

你好

UTF-8：

```
E4 BD A0 E5 A5 BD
```

它可以处理任意文本，但单个字符可能需要多个字节。

4. 混合方案

现代 LLM 往往使用：

```
Byte-level + BPE
```

或者：

```
Unicode-aware + BPE
```

还有一些新模型采用更特殊的设计，例如字节级、字符级和其他压缩策略的组合。

八、不同 LLM 使用什么 Tokenizer？

模型	常见 Tokenizer
GPT-2	Byte-level BPE
GPT-3	GPT-2 风格 Byte-level BPE
BERT	WordPiece
T5	SentencePiece Unigram
Llama 2	SentencePiece BPE
Llama 3	基于 tiktoken 的 Byte-level BPE 变体
Qwen2	基于 BPE 的 Tokenizer
DeepSeek 系列	不同版本采用不同 Tokenizer，需看具体模型

这里的 `BPE`、`WordPiece`、`Unigram` 是核心算法；`SentencePiece`、`tiktoken` 等是实现框架或 Tokenizer 工具。

九、Tokenizer 对 LLM 有什么影响？

1. Token 数量影响成本

同一段文本，不同 Tokenizer 可能得到不同数量的 Token。

英文：
```
The quick brown fox jumps over the lazy dog.
```

中文：
快速的棕色狐狸跳过了懒狗。

中文、英文、代码、数学公式的 Token 数量可能有明显差异。

2. 影响上下文窗口

如果模型支持 128K Tokens，那么输入文本被切分后必须在这个限制内。

3. 影响模型训练效率

Tokenizer 决定文本压缩效率：

```
相同文本长度
      ↓
不同 Tokenizer
      ↓
不同 Token 数量
      ↓
不同训练计算成本
```

4. 影响中文和代码能力

Tokenizer 对中文、英文、代码的切分效率不同，会影响模型学习和推理时的 Token 使用效率。

十、你应该重点学习哪几个？

如果你的目标是 LLM 应用开发、RAG、AI Agent，建议按这个顺序：

BPE：理解 Tokenizer 的核心原理。

Byte-level BPE：理解 GPT 系列和多语言 Tokenization。

WordPiece：理解 BERT 及传统 NLP。

Unigram + SentencePiece：理解 T5、Llama 等模型。

Tokenizer 实战：使用 Hugging Face Transformers 和 tiktoken 统计 Token 数量。

推荐实践

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(
    "bert-base-chinese"
)
text = "你好，世界！Hello LLM"
tokens = tokenizer.tokenize(text)ids = tokenizer.encode(text)
print("Tokens:", tokens)print("IDs:", ids)print("数量:", len(ids))
```

最核心的理解:

> Tokenizer 决定文本如何转换成 Token；LLM 决定这些 Token 的语义和下一 Token 概率。Tokenizer 不是大模型本身，但它是 LLM 输入输出的基础。

参考：gpt4 tokenizer：tiktoken GitHub 公开项目